ClaimSphere

Technical Design & Implementation Report

Senior Application Engineer (Senior UI Developer) — Case Study Submission · Reference 139644

A claims-adjudication workspace for ABC Insurance: a 25,000-record claims grid with role-based access, and a document workspace that opens, edits, splits, merges, comments on and annotates documents of **100 MB to 1 GB**.

Item	Details
Candidate	Rangaprasad
Date	9 November 2026
Source code	https://github.com/rranga/claimsphere
Live demo	https://rranga.github.io/claimsphere/ (GitHub Pages)
Stack	React 19 · TypeScript (strict) · Vite · Tailwind CSS v4 · TanStack Query · Redux Toolkit · TanStack Virtual · pdf.js · Radix UI · MSW · Vitest · Playwright · axe-core

Contents

1. Executive summary 3
Key results 3
2. Scope, inputs and assumptions 4
2.1 Inputs 4
2.2 Assumptions 4
2.3 Requirements traceability 4
3. The solution at a glance 6
4. Architecture 9
4.1 System context 9
4.2 Technology choices 9
4.3 Code organisation and boundaries 10
4.4 State management 10
4.5 Backend API contract 11
5. Key flows 12
5.1 Grid data flow 12
5.2 Grid to document workspace 12
5.3 Loading very large documents 12
5.4 Editing and saving documents 13
6. Design patterns used 14
7. Performance and scalability 15
Trade-offs considered 15
Scalability 16
8. Security and role-based access 17
8.1 Personas 17
8.2 Where enforcement lives 17
8.3 Further safeguards 17
9. Accessibility (WCAG 2.1 AA) 18
9.1 Verification 18
9.2 What was implemented 18
10. Coding standards and best practices 19
11. Testing and continuous integration 19
12. Running the project 21
Suggested five-minute walkthrough 21
13. Limitations and roadmap 21

1. Executive summary

ABC Insurance is replacing a legacy claims-processing system. Claims arrive by email, SFTP, portal and API, and each claim carries large documents that adjusters must review and reorganise. The case study asked for a scalable, high-performance web application covering a large data grid, role-based access, very large documents and document operations, together with a discussion of architecture, performance and trade-offs.

ClaimSphere is a working implementation of that brief, not a mock-up. Every functional and non-functional requirement is implemented and covered by automated tests. The highlights:

- **25,000 claims, instant scrolling.** Only about 20–35 rows exist on the page at any time, however many claims match. Sorting, filtering and search run on the server.
- **1 GB documents open in under 2 seconds.** The browser downloads only the parts of the file it needs: 6.6 MB (0.7%) to show the first page of a 1 GB document.
- **Security decided by the server.** Each role (Adjuster, Supervisor, Auditor, Admin) sees different records and actions. The interface adapts, but every request is checked again on the server.
- **Safe document editing.** Split, merge, reorder, rotate and delete with undo/redo. Saving is all-or-nothing, with progress, cancel and safe retry. Unsaved work survives a browser refresh.
- **Quality evidence.** 45 unit/integration tests, 17 end-to-end browser tests, zero lint warnings, and an automated WCAG 2.1 AA accessibility audit with no violations on every main screen.

Key results

Measure	Result
Claims in the grid	25,000 (server-side sort, filter, search)
Rows in the page at any scroll position	~20–35 (virtualized)
1 GB document: time to first page	< 2 s, 6.6 MB downloaded
Jump to page 900 of 1,500	+9.8 MB (1.6% of the file in total)
Initial JavaScript (app code, gzipped)	~76 KB; the PDF engine (123 KB) loads only when a document is opened
Automated tests	45 unit/integration + 17 end-to-end, incl. WCAG audits of 6 screens and states — all passing
Accessibility	WCAG 2.1 A/AA automated audit (axe-core): 0 violations on sign-in, grid and workspace
Static analysis	TypeScript strict mode, ESLint with zero warnings, Prettier enforced in CI and pre-commit

2. Scope, inputs and assumptions

2.1 Inputs

- Job description: Senior Application Engineer / Senior UI Developer, Swiss Re L&H Reinsurance Engineering.
- Case study: “Senior UI Engineering Use Case — Problem Statement” (functional, non-functional and discussion requirements).
- Figma: “CRM Dashboard – Customers List” (layout, typography, colour palette, KPI cards, table and pagination styling).

2.2 Assumptions

Topic	Assumption and rationale
Document size	The brief shows “1500 MB” corrected to “100 MB” (strike-through). The design targets 100 MB–1 GB, and because nothing scales with file size it also holds beyond 1.5 GB.
Backend	No backend was provided. A mock server (Mock Service Worker) implements a realistic REST contract — authentication, authorization, versions, validation and asynchronous jobs — so the UI is built exactly as it would be against a real API.
Figma	The CRM “Customers” screen was adapted to the claims domain: KPI cards became claim statistics; the customer table became the claims grid; the visual language (Poppins, #5932EA, status pills, pagination) is preserved, including both sidebar states — expanded and minimized (icons only, remembered per user, minimized by default below 1280 px).
Scope of “edit”	Structural editing of documents: reorder, rotate and delete pages; split, merge, rename and delete documents; plus page comments and annotations. Changing page content (text editing, redaction) is out of scope; it would run on the server on an immutable copy, with the browser sending the instruction rather than the bytes.
Platform context	Intake channels (Email, SFTP, Portal, API) and the risk score are treated as data supplied by upstream services: shown, filterable and sortable, with a high-risk KPI. The ingestion pipeline and a dedicated risk-assessment panel sit outside the UI scope; new services plug in behind the backend-for-frontend contract.
Document format	PDF, the de-facto format after ingestion and OCR. Other formats would be normalised to PDF at ingestion.
Devices	Designed for adjuster workstations and fully responsive down to 390 px phones: icon-only navigation below 1024 px, claim cards instead of table rows, side panels as slide-in sheets.
Test data	25,000 claims generated deterministically (same data on every load). Sample PDFs are committed; 250 MB and 1 GB PDFs are generated by a script.

2.3 Requirements traceability

Each requirement from the case study, how it is met, and where it is verified.

Requirement	Implementation	Verified by
Grid: 20,000+ records, sort, filter, row actions (Edit / Delete / Assign)	Server-side sort/filter/search; virtualized grid with on-demand loading in 100-row blocks; Figma-style pagination as an alternative mode; Edit, Assign and Delete dialogs	E2E: grid, pagination, filter. API tests: server-side sort/filter
Pagination vs alternative, justified	Virtualization as default, pagination as toggle; decision record ADR-0001	docs/adr/0001
RBAC: backend is the source of truth	Server policy checks every request; data scoped by region; per-row allowedActions; 401/403/404 responses	API tests: scoping, 403 on UI bypass, 404 no-leak

Requirement	Implementation	Verified by
RBAC: frontend for UX (show / hide / disable)	Hide when the role can never act; disable with an explanation when this record blocks it; route guard	Component test (RowActions); E2E per persona
Row → document: smooth transition	Prefetch on hover intent (code, document list, first page); instant header; skeletons; animated route transition; Back restores the grid	Manual + E2E navigation
View 100 MB–1 GB documents	pdf.js in a Web Worker; custom HTTP Range loader; only visible pages rendered	Measured: 6.6 MB for first page of 1 GB
Edit, split, merge, delete; page comments; annotations	Command log with undo/redo; same rules on client and server; comments and annotations bound to stable page ids	Unit: operation engine, Redux slice. E2E: split + save, annotate, comment
Performance: minimal re-renders and memory	Memoized rows, structural sharing, selector subscriptions, canvas release, LRU caches, code splitting	Section 7 measurements
Scalability: records, document size, concurrent users and operations	Server row model; download cost depends on pages viewed, not file size; stateless API; versions, idempotency keys and a job queue for concurrent edits	Section 7 (Scalability)
UX: progress, errors, cancel / retry	Job progress bar, cooperative cancel, idempotent retry, per-region error boundaries, toasts, skeletons	E2E: failure → retry
Reliability: consistent state, partial failures	Atomic commit, optimistic concurrency (409), idempotency keys, draft persistence, orphan cleanup	API tests: atomicity, idempotency, 409

3. The solution at a glance

The screenshots below are taken from the production build of the application, driven by the same automated browser used for end-to-end testing.

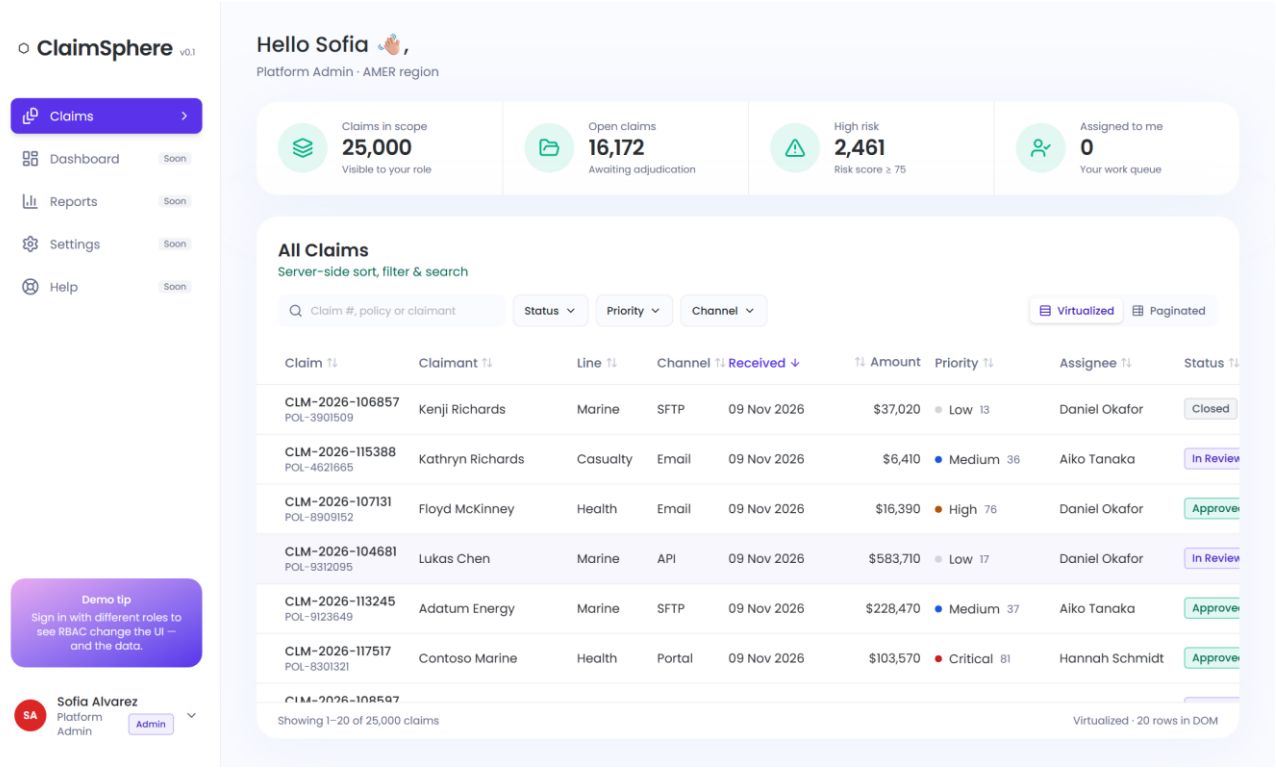


Figure 1 — Claims grid (Admin): KPI cards, server-side filters, virtualized grid of 25,000 claims; footer confirms only 22 rows are in the page.

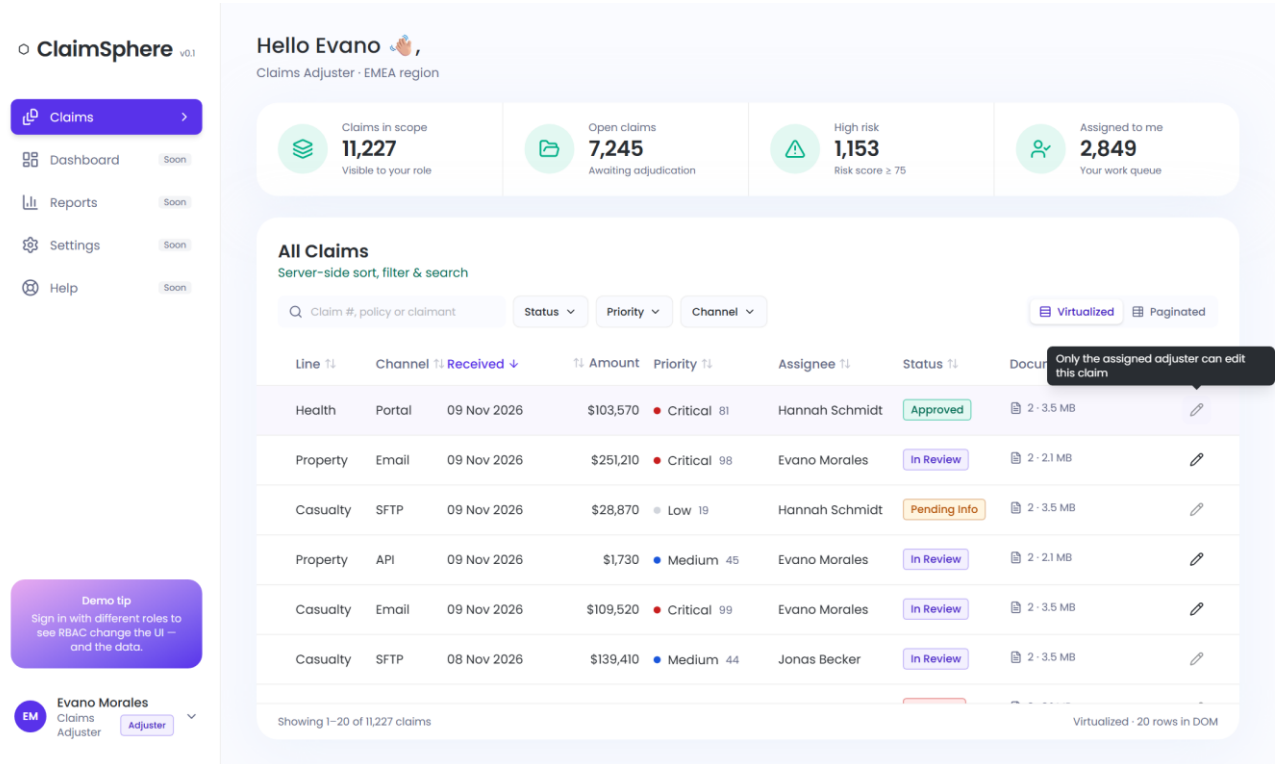


Figure 2 — Role-based UX (Adjuster, EMEA only): Edit is available on own claims and disabled with an explanation on colleagues' claims; Assign and Delete are hidden for this role.

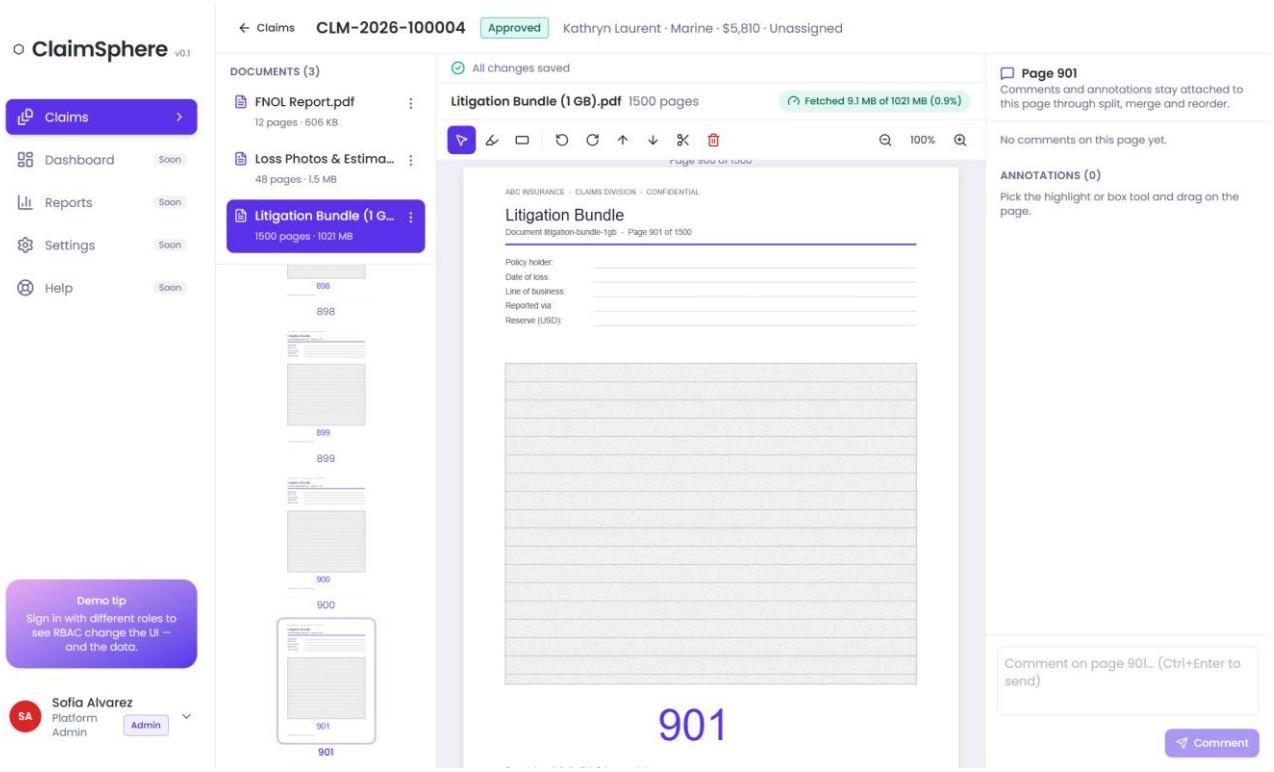


Figure 3 — 1 GB, 1,500-page document opened at page 901: only 9.1 MB (0.9%) has been downloaded (green badge).

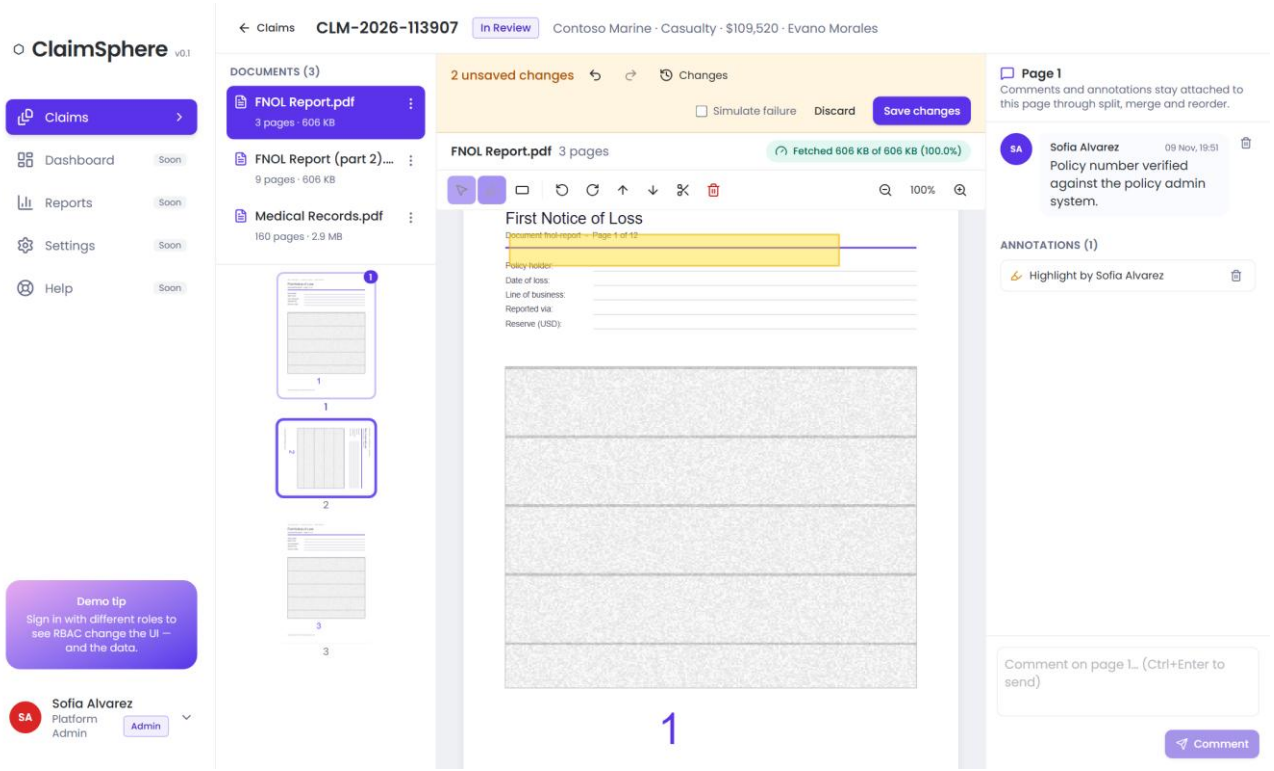


Figure 4 — Editing: document split into two, page rotated (draft with undo/redo), highlight annotation and a page comment.

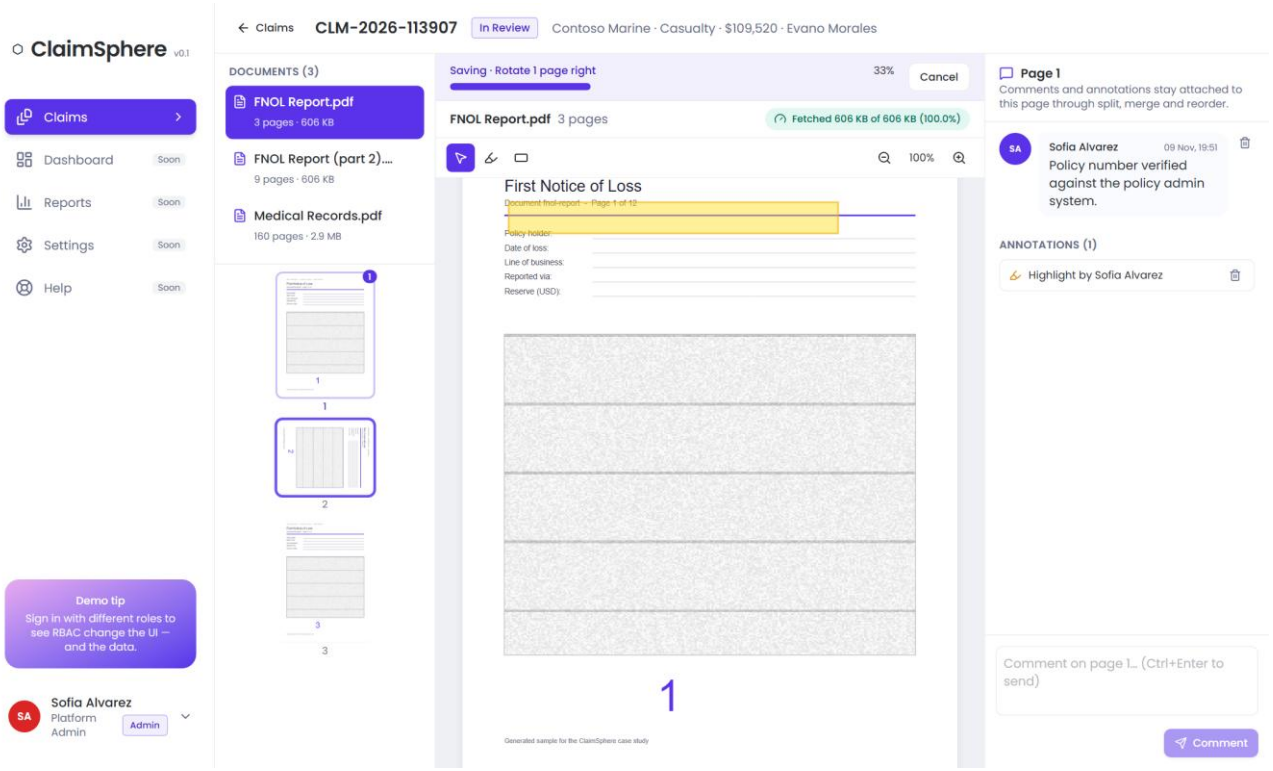


Figure 5 — Saving runs as a server job with live progress and Cancel; nothing is applied until the final atomic step.

4. Architecture

4.1 System context

The browser never downloads a whole document. It calls the API for data and commands, and fetches byte ranges of immutable source files directly from document storage (in production: object storage behind a CDN with short-lived signed URLs).

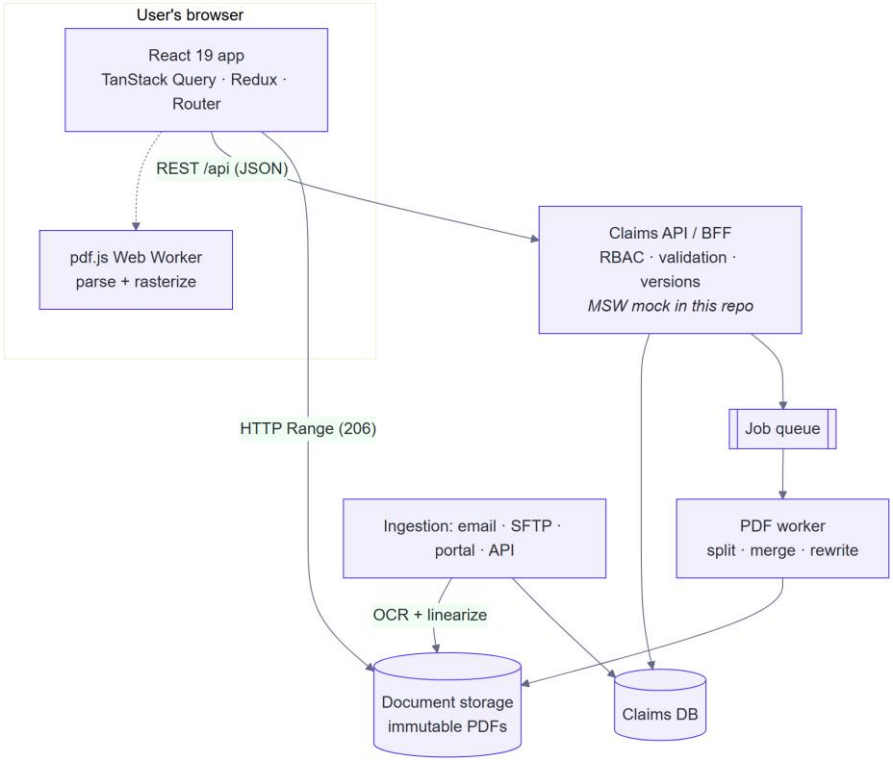


Figure 6 — System context

4.2 Technology choices

Concern	Choice	Why
UI framework	React 19 + TypeScript (strict)	Preferred in the job description; strict typing catches errors at build time
Build	Vite 7	Fast development server, native ES modules, simple code splitting
Styling	Tailwind CSS v4 with design tokens	Figma colours/typography defined once as tokens; consistent, small CSS (7 KB gzipped)
Accessible primitives	Radix UI (dialog, menu, tooltip)	Keyboard support, focus management and ARIA handled correctly
Server state	TanStack Query	Caching, request de-duplication, cancellation, prefetch, garbage collection
Client state	Redux Toolkit	Complex document-editing session with undo/redo
Virtualization	TanStack Virtual	Headless, works for both the grid and the page viewer
PDF rendering	pdf.js (Mozilla)	Industry-standard, renders in a Web Worker, supports partial loading
Mock backend	Mock Service Worker (MSW)	Real HTTP semantics in the browser and in tests; deployable as a static demo

Concern	Choice	Why
Testing	Vitest, React Testing Library, Playwright, axe-core	Unit, component, API-contract, end-to-end and accessibility

4.3 Code organisation and boundaries

The code is organised by feature, not by file type. Pages compose features; features never import each other's internals; shared code never depends on features. These rules are enforced by ESLint, so they cannot erode over time.

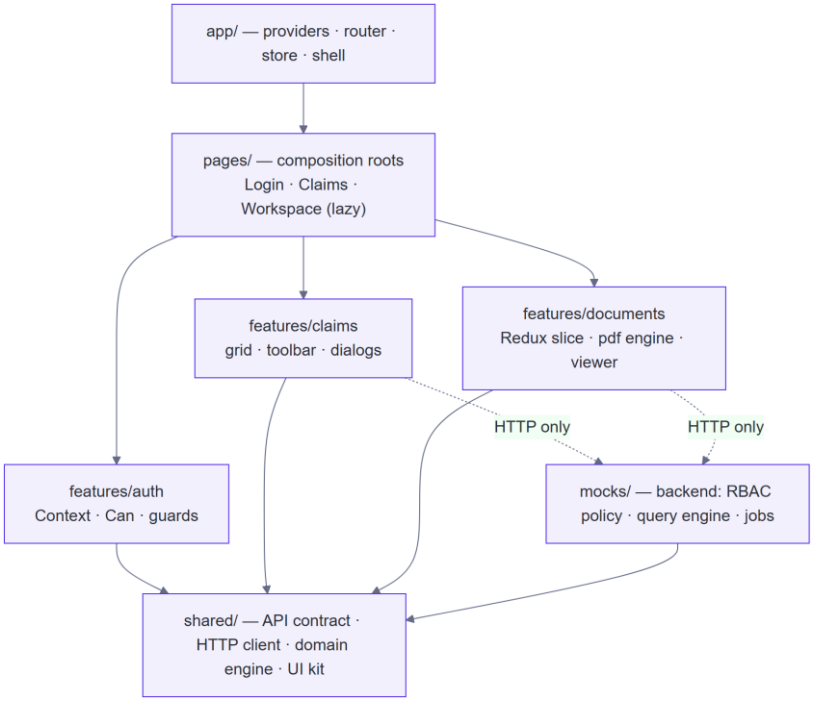


Figure 7 — Layers and dependency rules

Folder	Responsibility
`src/app`	Application shell: providers, router, Redux store, layout, error boundaries
`src/pages`	Route screens (Login, Claims, Workspace); the only place features are combined
`src/features/auth`	Session (React Context), `<Can>` permission gate, `usePermission`, route guard
`src/features/claims`	Grid (virtual and paged), toolbar, URL state, block loader, dialogs, mutations
`src/features/documents`	Workspace: Redux slice, PDF engine, viewer, thumbnails, save jobs, comments, annotations
`src/shared`	API contract (types), HTTP client, pure domain engine, UI components, utilities
`src/mocks`	The mock backend: data generator, RBAC policy, query engine, job engine
`docs/`	Architecture document and four Architecture Decision Records (ADRs)

4.4 State management

The job description lists Redux, Context API and MobX. Rather than one global store, each kind of state has exactly one owner, chosen for what that state needs:

State	Owner	Reasoning
Server data: claims, documents, comments, jobs	TanStack Query	Needs caching, de-duplication, cancellation, prefetch and eviction — reimplementing this in Redux adds code and bugs
Grid sort, filter, page, view mode	URL query string	Shareable links, survives refresh, Back button restores the exact view with no extra code
Document editing draft, selection, tool, zoom	Redux Toolkit	Client-owned, read by many distant components, updated frequently; the command log gives undo/redo and is the save payload
Signed-in user and permissions	React Context	Read everywhere but changes only at sign-in/out, so Context re-renders are irrelevant
PDF download statistics	`useSyncExternalStore`	The data lives in the PDF engine outside React; this is the tear-free way to read it
Transient UI (open menus, hover)	Component state	Local by nature

MobX was considered and not used: the combination above covers every case with explicit data flow and better debugging tools (Redux DevTools, Query DevTools).

4.5 Backend API contract

Shared TypeScript types (`src/shared/api/contracts.ts`) define the contract between UI and backend; in production these would be generated from an OpenAPI specification.

Endpoint	Purpose and behaviour
POST /auth/login · GET /auth/me	Session with user and role permissions
GET /claims?offset&limit&sortBy&sortDir&q&status&priority&channel	Server-side paging, sort, filter, search; each row includes a `version` and the actions the current user may perform
GET /claims/stats	KPI cards, restricted to the user's data scope
PATCH /claims/{id} · POST /claims/{id}/assign · DELETE /claims/{id}	Writes carry a version; stale writes are rejected with 409
GET /claims/{id}/documents	Document structure (page references), file metadata (size, page count, page size) and the user's capabilities
POST /claims/{id}/document-operations	Starts a save job (202 Accepted); idempotent per key; 409 on conflict, 422 on invalid operations
GET /jobs/{id} · POST /jobs/{id}/cancel	Job status/progress; cooperative cancellation
/claims/{id}/comments · /claims/{id}/annotations	Collaboration data keyed by stable page id
Document file URLs	Must support HTTP Range requests (206 Partial Content)

5. Key flows

5.1 Grid data flow

- The virtualizer reports which rows are visible. Only the 100-row blocks covering those rows (plus a margin) are requested.
- Each block is a separate cache entry: requests are de-duplicated, cancelled if no longer needed, and evicted 60 seconds after scrolling away, so memory stays flat no matter how far the user scrolls.
- The server caches the filtered and sorted list per user and query, so 250 block requests cost one sort, not 250.

5.2 Grid to document workspace

1. **Intent detection.** Hovering a row for 120 ms (or focusing it with the keyboard) starts prefetching in parallel: the workspace code, the document list, and the first page of the first document.
2. **Instant navigation.** The click navigates immediately with an animated transition; the header renders at once from the grid row data while skeletons hold the layout.
3. **Return.** Back restores the grid's filters, sort and page from the URL and its data from the cache — no reload flash.

5.3 Loading very large documents

pdf.js normally starts with a full download and aborts it once it sees the server supports partial requests. Because the server already reports the file size, a **custom Range transport** skips that step: every byte that crosses the network is an explicit, cancellable, counted request. Parsing and drawing happen in a Web Worker so the interface never freezes.

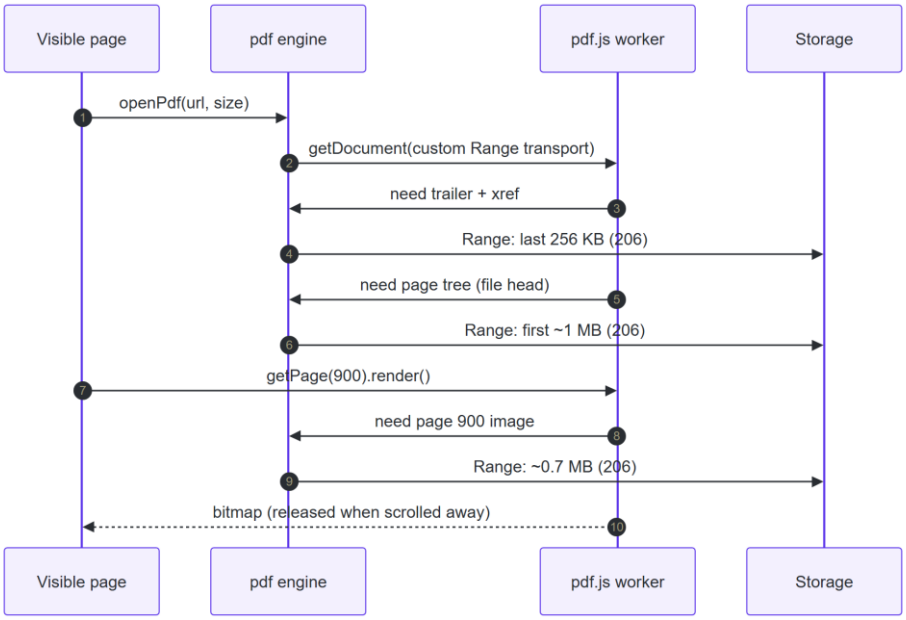


Figure 8 — Loading page 900 of a 1 GB document

Finding: the layout of the PDF matters as much as the viewer

Locating page N requires reading the PDF's page tree. In a file where each small page entry sits next to a large image, reaching page 360 of a 250 MB file forced the download of **152 MB**. With page entries stored together at the start of the file (a "web-optimised" layout), the same jump downloaded **~10 MB**.

Recommendation: the ingestion pipeline should web-optimise (linearize) every PDF after OCR. The included document generator produces this layout.

5.4 Editing and saving documents

A document is an ordered list of **page references** — (source file, page number, rotation) — never raw bytes. Split, merge, reorder, rotate and delete therefore take the same time for a 1 MB or a 1 GB file. Each edit is a command in a log; the visible document is the log replayed over the last saved version. The **same pure function** applies the log in the browser (instant preview) and on the server (authoritative save).

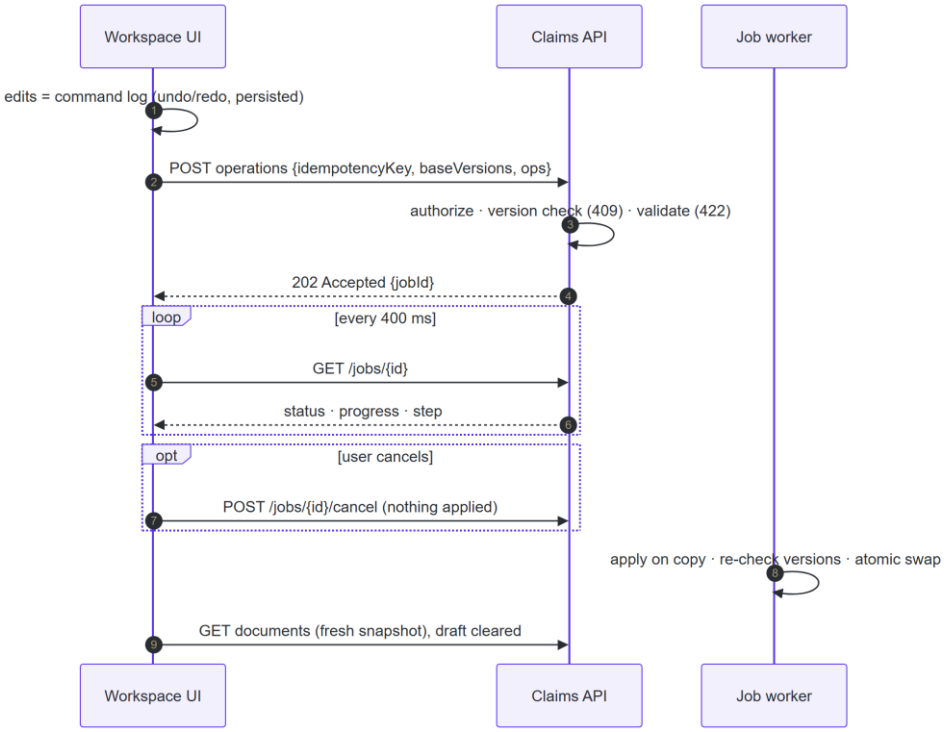


Figure 9 — Saving edits as one atomic, cancellable, idempotent job

Guarantee	How
All-or-nothing	The server applies all operations to a copy and swaps it in only at the final step; a failure or cancel leaves documents untouched — there are no partial failures to repair
No lost updates	The request includes the versions the user edited; if someone else saved first, the server returns 409 and the user chooses “keep my draft” or “load latest”
No duplicates	An idempotency key per save attempt: a double-click or network retry returns the same job
Safe retry	The draft is kept on failure, so Retry resubmits exactly the same work
No lost work	The command log is stored locally and restored after a refresh — only if the documents have not changed in between
Comments stay attached	Comments and annotations reference stable page ids, so they follow pages through split, merge and reorder; those on deleted pages are removed in the same save

6. Design patterns used

Pattern	Where	Benefit
Feature-based modular architecture	`features/*` with public `index.ts`; lint-enforced boundaries	Teams can own features independently; dependencies stay one-directional
Composition root	`pages/*` wire features together	Features stay decoupled (the grid does not know about the PDF engine)
Command pattern (event-sourcing style)	Document operations log in Redux	Undo/redo, replay, persistence and the save payload from one model
Functional core, imperative shell	`shared/domain/documentOperations.ts`	Pure, deterministic, unit-testable; identical rules on client and server
Adapter	`HttpRangeTransport` adapts HTTP Range requests to the pdf.js transport interface	Full control over network use, cancellation and measurement
API client layer + query-key factory	`claimsApi`, `documentsApi`, `claimKeys`, `queryOptions` factories	One place per endpoint; precise cache invalidation
Observer / external store	PDF download statistics + `useSyncExternalStore`	Live UI updates from a non-React source without global state
Cache with LRU eviction	Open PDF documents (max 4); grid blocks via cache GC	Bounded memory
Optimistic UI with rollback	Assign, comments, annotations	Instant feedback; consistent state on failure
Optimistic concurrency control	Version tokens on every write (409 on conflict)	No silent overwrites between users
Asynchronous request-reply	202 Accepted + job status endpoint	Long operations without blocking or timeouts
Idempotency key	Save job submission	Safe retries
Policy object + declarative guards	Server `policy.ts`; client `<Can>`, `RequireAuth`	Authorization rules in one place; readable UI code
Bulkhead error boundaries	Shell, viewer, thumbnails, comments	A failure in one region does not take down the page
Strategy	Virtual and paged grid share columns and rows	Two row models, one presentation
Lazy loading + prefetch on intent	Workspace route, dialogs, PDF engine	Small initial download, yet instant navigation
Headless components + design tokens	Radix primitives styled with Tailwind tokens	Accessible behaviour with a consistent, themeable look

7. Performance and scalability

Measured in Chromium on a development laptop against the development build (bundle sizes from the production build).

Area	Technique	Result
Grid rendering	Virtualization (TanStack Virtual), fixed 56 px rows, overscan	~20–35 rows in the page for 25,000 claims
Grid memory	100-row blocks as cache entries, evicted 60 s after leaving view	Flat memory; no growth while scrolling
Re-renders	Memoized rows, stable callbacks, structural sharing of query data, Redux selectors	Unchanged rows never re-render
Server load	Debounced search (300 ms); server caches sorted results; hard limit of 500 rows per request	One request per pause in typing
Perceived speed	Prefetch on hover intent; skeletons; instant header from grid data; previous page kept visible while the next loads	Workspace opens with content ready
Document download	Custom Range loader, 256 KB chunks, no background download	1 GB file: first page < 2 s with 6.6 MB
Document rendering	Web Worker; virtualized pages; render only after 60 ms on screen; cancel when scrolled away	Fast scrolling does not queue hundreds of renders
Document memory	Release canvas memory when a page leaves view; cap pixel density at 2×; keep at most 4 documents open	Off-screen pages hold no images
Bundle size	Lazy workspace route and dialogs; PDF engine in its own chunk; vendor chunks for caching	App code ~76 KB gzipped (down from 168 KB before splitting)
Annotations	Vector overlay in normalized coordinates	Zoom and rotation never redraw the PDF for annotations
Responsive layout	Icon rail and claim cards below 1024 px; panels as sheets on phones; pinned Actions column	No horizontal overflow from 390 px phones up

Trade-offs considered

Decision	Alternatives	Why this choice
Virtualization + server blocks (pagination available)	Pagination only; infinite scroll; load all rows	Constant DOM and memory, true scrollbar, jump to any row; infinite scroll grows without bound and cannot jump
Server-side sort and filter	Client-side (20k rows is feasible today)	Scales with growth and respects data scoping — the client must never receive rows the user may not see
pdf.js with Range requests	Server-rendered page images/tiles	No rendition pipeline or storage; text stays selectable. Server tiles remain a good option for weak devices (hybrid possible)
Pessimistic, atomic save for document structure	Optimistic save	Expensive, destructive operations on large files must be all-or-nothing and truthful to the user
Optimistic comments, annotations, assignment	Pessimistic	Cheap, reversible, frequent — instant feedback matters more; rollback on failure
Polling job status (400 ms)	Server-Sent Events / WebSocket	Simple and robust for a demo; production would push updates

Scalability

The case study asks for growth in record volume, document size, and concurrent users and operations. None of them changes the client's cost per screen.

Growth	Why the design holds	Next step at much larger scale
Record volume (20k → millions)	Sort, filter and search run on the server; the client holds only the blocks in view; the server caps page size at 500	Indexes on sort/filter columns; keyset (cursor) paging; a search index for free text
Document size (1 GB and beyond)	Only the byte ranges of visible pages are downloaded; split, merge and reorder change page references, never bytes	Linearize at ingestion; object storage behind a CDN; server tiles for weak devices
Concurrent users	Stateless API (bearer token, no server session) scales horizontally; immutable, versioned files are CDN-cacheable and shared by all viewers	Autoscale the backend-for-frontend; cache per-user query results in Redis
Concurrent operations	Versions (409 on conflict), idempotency keys and atomic jobs: no silent overwrites, no double saves; long work runs on a queue, not in the request	Scale queue workers independently; push job progress over SSE/WebSocket

8. Security and role-based access

8.1 Personas

Persona	Role	Sees	Can do
Evano Morales	Adjuster	EMEA claims	Edit own claims; edit documents on own claims; comment; annotate
Priya Raman	Supervisor	EMEA claims	Edit and assign any open claim in the region
Marcus Lee	Viewer (auditor)	All regions	Read only
Sofia Alvarez	Admin	All regions	Everything, including delete

8.2 Where enforcement lives

Concern	Server (authoritative)	Client (user experience)
Authentication	Bearer token checked on every request; 401 signs the user out	Route guard; session restore
Authorization	Role permissions, region scoping, ownership and closed-claim lock — re-checked on every endpoint	Hide actions the role can never perform; disable with a reason when the record prevents it
Data scoping	Records outside the user's scope return 404 (existence is not revealed)	—
Validation	Schemas (zod) on every write; document operations dry-run before a job is accepted	Same rules for instant feedback; URL parameters whitelisted
Consistency	Version tokens (409), idempotency keys, atomic commit	Conflict dialog; draft kept on failure

8.3 Further safeguards

- No raw HTML rendering; comment text is always rendered as text (prevents cross-site scripting).
- PDF engine runs with XFA forms disabled; documents are parsed in an isolated worker.
- All query parameters are validated against allow-lists before reaching the API; the server caps page sizes.
- Production hardening (documented, not in the demo): OIDC single sign-on with httpOnly cookies, Content Security Policy, short-lived signed document URLs, audit logging of document operations.

9. Accessibility (WCAG 2.1 AA)

Accessibility was treated as a requirement, not a final check: every primary screen is audited automatically, and the build fails on any violation.

9.1 Verification

- **Automated audit:** axe-core runs in Playwright against the sign-in screen, the claims grid, the document workspace, the minimized sidebar and the phone layouts of the grid and workspace with the WCAG 2.0/2.1 A and AA rule sets — **0 violations**.
- **Static analysis:** eslint-plugin-jsx-a11y on every commit (labels, roles, keyboard handlers, no autofocus).
- **Colour contrast:** the Figma greys and greens were too light for small text (as low as 2.8:1). They were darkened within the same hue so that every text colour meets **4.5:1** on every surface it is used on.

9.2 What was implemented

WCAG criterion	Implementation
1.3.1 Info and relationships	Semantic roles: grid / row / columnheader / gridcell, listbox / option for thumbnails, toolbar, radiogroup, navigation and region landmarks, figure / figcaption for pages; labelled form fields
1.4.3 Contrast (minimum)	All text tokens ≥ 4.5:1 (verified by axe-core)
2.1.1 Keyboard	Grid: arrows, Page Up/Down, Home/End, Enter opens. Thumbnails: arrows, Shift to extend selection, Space/Enter to select, Alt+arrows to move a page. Ctrl+Z / Ctrl+Shift+Z undo/redo. Menus and dialogs fully keyboard operable
2.4.1 Bypass blocks	“Skip to content” link
2.4.3 Focus order	Roving tab index: the grid and the thumbnail list are each a single tab stop
2.4.7 Focus visible	High-contrast focus ring on every interactive element
3.3.1 / 3.3.2 Errors and labels	Inline errors linked to fields (aria-invalid, aria-describedby)
2.5.3 Label in name	Every accessible name contains the visible label
4.1.2 Name, role, value	aria-sort on columns, aria-selected, aria-pressed, aria-checked, aria-busy, progressbar values; disabled actions stay focusable with an explanation (aria-disabled)
4.1.3 Status messages	Live regions for result counts, save progress (status) and errors (alert); toast notifications
Minimized navigation	Icon-only links keep accessible names and show tooltips on hover and focus; toggle exposes aria-expanded
Virtualized content	aria-rowcount / aria-rowindex so assistive technology knows the grid has 25,000 rows even though only ~30 are rendered
Motion	prefers-reduced-motion disables animations and route transitions
Drag and drop alternative	Page reordering by drag also available via buttons and Alt+arrow keys (WCAG 2.2 “Dragging movements”)

Known gaps (honest assessment)

• Page content is drawn on a canvas, so screen-reader users cannot read the document text yet. **Fix:** add pdf.js' text layer (also enables text selection and search).

• Annotations can only be drawn with a pointer. **Fix:** a keyboard "add annotation to selected region" command.

• No manual testing with screen readers (NVDA, JAWS, VoiceOver) has been done; automated tools detect only part of WCAG issues. Recommended before production.

10. Coding standards and best practices

Area	Practice
Type safety	TypeScript strict mode plus `noUncheckedIndexedAccess`, `noImplicitOverride`, `verbatimModuleSyntax`; no `any` in application code; one shared API contract
Linting	ESLint (flat config): TypeScript rules, React Hooks rules including React Compiler checks, jsx-a11y, consistent type imports; zero warnings allowed
Architecture rules	Lint rules forbid shared code importing features and UI code importing the mock backend
Formatting	Prettier with Tailwind class sorting; checked in CI
Git hygiene	Husky pre-commit hook runs ESLint and Prettier on staged files; Conventional Commit messages
Components	Function components and hooks; small, single-purpose files; one dialog host per page rather than per row; memoization where it measurably matters
Design system	Design tokens from Figma in one place; reusable primitives (Button, Badge, Dialog, Tooltip, Field, Skeleton, ProgressBar, ErrorBoundary)
Error handling	Typed API errors; transient errors retried with exponential back-off, client errors never retried; error boundaries per region
Documentation	Comments explain why, not what; README with requirement map; architecture document; four decision records (ADRs)
Reproducibility	Pinned Node version (.nvmrc), lock file, deterministic test data, `npm run validate` single command

11. Testing and continuous integration

Level	Tool	What is covered	Count
Unit	Vitest	Document operation engine; Redux slice (undo/redo, selection, draft restore); annotation geometry; URL parsing; pagination; LRU cache	29
API contract	Vitest + MSW	Authentication, authorization (403), data scoping (404), per-row actions, server sort/filter, 409 conflicts, atomic + idempotent jobs, 422 validation	13
Component	React Testing Library	Hide / disable rules for row actions per role	3
End-to-end	Playwright (Chromium)	Virtualization, filtering, RBAC per persona, pagination, split + save, failure + retry, annotations + comments, read-only workspace, sidebar states, short screens, phone and tablet-portrait layouts	13

Level	Tool	What is covered	Count
Accessibility	Playwright + axe-core	WCAG 2.1 A/AA on sign-in, grid, workspace and the minimized sidebar (phone layouts are audited inside the responsive tests)	4

Continuous integration (GitHub Actions) runs on every push and pull request: format check, lint, type check, unit tests, production build and the end-to-end suite. A second workflow deploys the live demo to GitHub Pages, generating a 250 MB document at deploy time so partial loading can be seen live.

12. Running the project

Command	Purpose
`npm install`	Install dependencies (Node 22.18 or later)
`npm run dev`	Start the application at http://localhost:5173
`npm run generate:docs -- --preset large`	Generate 250 MB and 1 GB sample documents (about 2 seconds)
`npm run validate`	Lint, type check and unit/integration tests
`npm run test:e2e`	End-to-end and accessibility tests
`npm run build`	Production build

Suggested five-minute walkthrough

1. Sign in as **Sofia (Admin)**. Drag the grid scrollbar to the middle of 25,000 claims; the footer shows how few rows are in the page. Switch to **Paginated** for the Figma layout.
2. Filter by Status and sort a column; note the URL updates and the Back button restores it.
3. Sign out and sign in as **Marcus (Auditor)**: no row actions. Then **Evano (Adjuster)**: only EMEA claims; Edit disabled with a reason on colleagues' claims.
4. As Sofia, search for **100004**, open the claim and select the **Litigation Bundle (1 GB)** (generated locally; the live demo includes a 250 MB document on every fourth claim). Watch the green download badge, then jump deep into the document.
5. Open an In-Review claim: split a document, rotate a page, undo with Ctrl+Z, then **Save** to watch the job. Tick **Simulate failure** to see the atomic rollback and safe Retry.
6. Refresh the page mid-edit: the draft is restored. Add a highlight and a page comment.

13. Limitations and roadmap

Current state	Next step for production
Mock backend runs in the browser; data resets on reload	Node or .NET backend-for-frontend implementing the same contract (OpenAPI-generated types)
Save job simulates PDF rewriting	Queue worker (e.g. qpdf/PDFium) writing new immutable file versions
Job status by polling	Server-Sent Events or WebSocket push; live collaboration (presence, live annotations)
Demo sign-in with a session token	OIDC single sign-on (e.g. Azure AD), httpOnly cookies, CSP
Document text not exposed to screen readers	pdf.js text layer: accessibility, selection and in-document search
Touch: annotations work with touch, but there is no pinch-to-zoom	Pinch-to-zoom and offline support (PWA) for field adjusters
No visual component catalogue	Storybook for the UI primitives; visual regression tests
Client observability	Error and performance telemetry (e.g. Sentry, Web Vitals)

Detailed decision records are in the repository under `docs/adr`: virtualized grid (0001), large-document loading (0002), state management (0003) and optimistic vs pessimistic updates (0004).